# Building a RAG Voice Assistant with LiveKit and Dify

Project journey, troubleshooting process, and final resolution

This report documents the development path, the approaches that did not meet the requirements, the errors encountered, and the changes that finally made the LiveKit voice assistant retrieve answers from the intended Dify knowledge base.

Project troubleshooting report
19 October 2026

# 1. Project Goal

The goal was to build a voice assistant in LiveKit Agent Builder that could answer questions using a PDF stored in a Dify knowledge base. The assistant was intended to rely on retrieved document content rather than answer questions about the PDF from general knowledge. The main source document was the PDF about AI agents, harnesses, and tools.

The desired flow was: the user asks a question in LiveKit; LiveKit calls a tool; Dify retrieves relevant chunks from the selected knowledge base; and the assistant uses those chunks to formulate an answer.

# 2. First Approach — Building a Dify Workflow

The first approach was to create a workflow in Dify and connect that workflow to LiveKit. This approach worked in the sense that the assistant could communicate with the workflow and receive responses. However, retrieval was too sensitive to the exact wording of a question.

For example, if a question contained a spelling mistake or a word was not capitalized in the same way as it appeared in the knowledge base, the assistant sometimes failed to return the expected answer. A case variation such as “Codex” versus “codex” was one example of the kind of issue encountered.

This made the first approach unreliable for normal voice-assistant use. People may phrase the same question differently, misspell a word, or use different capitalization. The system needed to handle natural variations more consistently.

# 3. Second Approach — Trying a Dify Agent Application

After the workflow approach showed these limitations, a Dify Agent application was tried instead. The intention was to use the agent as the main component and avoid depending on the original workflow setup.

This approach did not work as intended. The assistant still could not reliably retrieve and provide the required answers, so the project moved to a more direct integration between LiveKit and the Dify knowledge base.

# 4. Third Approach — Connecting the Knowledge Base Directly to LiveKit

The next approach was to bypass the Dify workflow and agent application and connect LiveKit directly to the Dify Knowledge API. In LiveKit Agent Builder, an HTTP action named **ask_dify** was configured to send the user's question to Dify's dataset retrieval endpoint.

The tool used the POST method and a required string parameter named **query**, intended to contain the complete question. An Authorization header was configured with a Dify dataset API key. The intended endpoint format was:

**https://api.dify.ai/v1/datasets/{dataset_id}/retrieve**

This was a simpler architecture: LiveKit sent the question to the knowledge retrieval endpoint, and Dify returned matching chunks from the selected dataset.

# 5. Errors Encountered and What They Meant

During the direct integration, several HTTP errors appeared. They were not all the same problem, and each one needed to be interpreted separately.

### HTTP 403 — Forbidden

The error message stated that the API key was not authorized to access the knowledge base. This initially suggested an API-key permission problem. The Dify API Access setting was checked, and the Authorization header and dataset API key were reviewed. Those repeated checks did not resolve the issue.

### HTTP 404 — Not Found

A 404 indicated that the requested resource or endpoint could not be found. In this troubleshooting process, it was important to check that the endpoint path and dataset ID referred to the intended knowledge base, rather than assuming that all errors were caused by the API key.

### HTTP 400 — Bad Request / Invalid Parameter

A 400 indicated that the request was rejected because of a parameter or configuration problem. During troubleshooting, a separate error message also referred to incorrect model credentials. That message could relate to model credentials used by a component in the retrieval path, so it needed to be distinguished from the dataset API authorization error.

The errors were confusing because they appeared at different stages and did not all point to the same root cause. Repeating the same API-key checks was not enough; the exact request endpoint and target dataset had to be compared.

## 6. Finding the Actual Dataset Mismatch

The Dify Knowledge page showed two knowledge bases with very similar AI_Agents_Harnesses names. This made it possible to inspect one dataset while LiveKit was requesting another.

The dataset ID visible in the intended Dify knowledge base's browser URL was:

**4cf1e0c7-c870-4dd3-94fc-fcbe7f426a40**

The dataset ID originally configured in LiveKit was:

**30774a7e-a4d7-43b3-bec1-451257689739**

These IDs were different. That mismatch was the key discovery: the LiveKit request was not targeting the same knowledge base that was being inspected in Dify. The names looked similar, but the unique dataset IDs did not match.

## 7. Final Fix

The dataset ID in LiveKit's URL was changed to the ID of the intended Dify knowledge base. The corrected endpoint was:

**https://api.dify.ai/v1/datasets/4cf1e0c7-c870-4dd3-94fc-fcbe7f426a40/retrieve**

The POST method, required **query** parameter, and Dify dataset API Authorization header were retained. The tool was updated and tested again. The user then confirmed that it worked.

**The decisive fix was to point LiveKit at the correct dataset ID. Rebuilding the workflow was not required for the final working direct-knowledge-base approach.**

## 8. Final Architecture

- **User:** asks a question by voice in LiveKit.
- **LiveKit Agent Builder:** invokes the HTTP action named ask_dify.
- **Dify Knowledge API:** receives the query and searches the intended dataset.
- **Dify response:** returns relevant document chunks.
- **LiveKit assistant:** uses the retrieved material to respond to the user.

The final path avoided the original workflow and agent application and used direct retrieval from the knowledge base.

## 9. Lessons Learned

- **Compare unique IDs, not just names.** Similar dataset names can hide a mismatch; confirm the dataset ID in the Dify page against the one in LiveKit's URL.
- **Separate the different API keys.** A model-provider key, such as a Gemini key, is not the same as a Dify dataset API key.
- **Read the exact error.** A 403 authorization error, a 404 resource error, and a 400 invalid-request or credentials error can have different causes.
- **Test realistic questions.** Check spelling variations, capitalization, and alternative phrasings when evaluating retrieval quality.
- **Change one thing at a time.** Compare the endpoint and dataset ID before repeatedly changing parameters or credentials.
- **Keep credentials private.** Never place API keys in a report, public screenshot, or source repository.

## 10. Final Configuration Summary

| Setting | Final value / status |
|---|---|
| Platform | LiveKit Agent Builder + Dify Knowledge API |
| HTTP action | ask_dify |
| Method | POST |
| Endpoint | https://api.dify.ai/v1/datasets/4cf1e0c7-c870-4dd3-94fc-fcbe7f426a40/retrieve |
| Request parameter | **query** — required string containing the user's question |
| Authorization | Dify dataset API key in the Authorization header, formatted as Bearer YOUR_KEY |
| Outcome | Working after correcting the dataset ID |

## 11. Conclusion

The project progressed through three approaches: a Dify workflow connected to LiveKit, a Dify Agent application, and finally direct connection to the Dify Knowledge API. The workflow initially communicated but struggled with spelling and capitalization variations. The agent approach did not work as intended. The direct integration then exposed several different errors, including 403, 404, and 400 responses. Comparing the exact dataset IDs revealed that LiveKit was targeting a different dataset from the one intended. Correcting that ID resolved the issue, and the user confirmed that the integration worked.